Dictionaries and Associative Data

Lecture 25

CS 1001

Kirk Duran

Today

- Explain why sequential position is not always the natural way to organize data.
- Model a dictionary as a collection of **key–value associations**.
- Create dictionaries, retrieve values, add new associations, and update existing ones.
- Control early exits and skips, preserve progress, and traverse collections safely.
- Iterate over keys, values, and pairs; keep the best entry seen so far.
- Combine iteration and dictionaries to construct a character-frequency map.

Key idea

Dictionaries change how data is identified; loop-control statements change how repeated processing proceeds.

The Boundary for This Lecture

Today

- dictionary literals
- keys and values
- lookup by key
- adding and updating entries
- membership
- loop control and safe traversal
- dictionary iteration and best-so-far state
- frequency maps

Not today

- hash-function design
- collision-resolution algorithms
- asymptotic complexity analysis
- nested dictionaries
- large catalogs of dictionary methods
- implementing a hash table from scratch

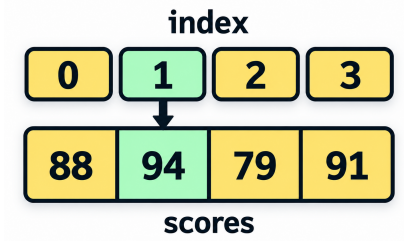
Part 1: Why Associative Data?

Sequences Organize Values by Position

A list

```
scores = [88, 94, 79, 91]
```

- Each value is identified by an integer index.
- Position is useful when order itself matters.
- The natural question is: **which position do I want?**



Sometimes Position Is the Wrong Question

Suppose each score belongs to a particular student.

Parallel information

```
names = ["Ada", "Grace", "Linus", "Edsger"]
scores = [88, 94, 79, 91]
```

To find Grace's score, we must first determine Grace's position and then use the same position in `scores`.

name $\longrightarrow$ **position** $\longrightarrow$ **score**

Key idea

The data relationship we care about is not really "index 1 has value 94." It is "Grace has score 94."

Associative Data Stores the Relationship Directly

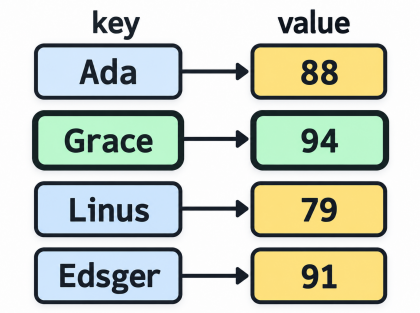
Dictionary

```
scores = {"Ada":  88, "Grace":  94,
          "Linus":  79, "Edsger":  91}
```

Now the lookup relationship is direct:

$$\texttt{"Grace"} \longrightarrow 94$$

The name is not merely stored as another value. It identifies the associated score.



Definition: Dictionary

Definition

A **dictionary** is a mutable mapping from **keys** to **values**. Each key identifies one associated value.

$$\textbf{key} \longrightarrow \textbf{value}$$

- A key is used to identify an association.
- A value is the data stored for that key.
- The complete association is often called a **key–value pair**.

Anatomy of a Dictionary

Example

```
inventory = {"bolts":  120, "washers":  75, "nuts":  90}
```

Key	Associated value
"bolts"	120
"washers"	75
"nuts"	90

Key idea

A dictionary is organized by association, not by numerical position.

Dictionary Keys Are Unique

A dictionary cannot contain two independent entries for the same key.

Conceptual state

`"Ada"` $\rightarrow$ 88
`"Grace"` $\rightarrow$ 94

If we later associate `"Ada"` with 95, the existing association is updated.

`"Ada"` $\rightarrow$ 95

Key idea

One key identifies one current value.

Dictionary Properties

For our purposes, a Python dictionary has four important properties.

1. It stores **key–value associations**.
2. Each key is **unique** within the dictionary.
3. It is **mutable**: associations can be added or updated.
4. Values are retrieved using a **key**, not a positional index.

Mental-model shift

Do not ask "what is element 2?" Ask "what value is associated with this key?"

Part 2: Creating, Reading, and Updating Dictionaries

Creating Dictionaries

Dictionary literal

```
ages = {"Ada":  36, "Grace":  85, "Alan":  41}
```

Empty dictionary

```
counts = {}
```

- A colon separates each key from its value.
- Commas separate key–value pairs.
- An empty pair of braces creates an empty dictionary.

Retrieving a Value by Key

Dictionary lookup

```
ages = {"Ada":  36, "Grace":  85, "Alan":  41}
age = ages["Grace"]
```

The lookup expression evaluates to the associated value:

$$\texttt{ages["Grace"]} \longrightarrow 85$$

Key idea

Square brackets still mean “retrieve something,” but the selector is now a key rather than a positional index.

Membership Tests Keys

The `in` operator asks whether a key is present.

Example

```
ages = {"Ada":  36, "Grace":  85}

"Ada" in ages ⟶ True
"Alan" in ages ⟶ False
```

Important

For a dictionary, `in` tests **keys**, not values.

Guarding a Lookup

Looking up a key that is not present causes a `KeyError`.

Safe pattern

```
if name in ages:
   age = ages[name]
```

This uses a familiar conditional structure:

check whether the key exists $\longrightarrow$ retrieve only when valid

We will use this same idea when constructing a frequency map.

Adding a New Association

Assignment through a new key adds a key–value pair.

Example

```
ages = {}
ages["Ada"] = 36
ages["Grace"] = 85
```

Afterwards:

$$\texttt{ages} \rightarrow \texttt{\{"Ada": 36, "Grace": 85\}}$$

Key idea

Dictionary assignment binds a value to a key inside the dictionary.

Updating an Existing Association

Assignment through an existing key replaces the old associated value.

Example

```
scores = {"Ada":  88, "Grace":  94}
scores["Ada"] = 95
```

Afterwards:

$$\texttt{scores} \rightarrow \texttt{\{"Ada": 95, "Grace": 94\}}$$

No second `"Ada"` entry is created.

Add or Update Depends on the Key

The same syntax has two possible effects.

Statement	Effect
`d[new_key] = value`	add a new association
`d[existing_key] = value`	update the existing association

Key idea

The dictionary remains a mapping with one current value per key.

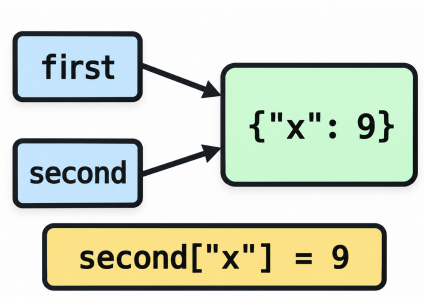
Dictionaries Are Mutable

Aliasing example

```
first = {"x":  1}
second = first
second["x"] = 9
```

Afterwards, both names refer to the same changed dictionary.

`first["x"]` $\longrightarrow$ 9



Length Means Number of Associations

Example

```
inventory = {"bolts":  120, "washers":  75, "nuts":  90}
```

Then:

$$\texttt{len(inventory)} \longrightarrow 3$$

The length is the number of key–value pairs, not the sum of the values and not the largest possible key.

What Can Be a Key?

Dictionary keys must be suitable for stable identification.

- Strings are common keys: `"Ada"`, `"banana"`, `"CA"`.
- Integers can also be keys: `101`, `202`, `404`.
- Mutable lists cannot be dictionary keys.

For this course

We will primarily use immutable primitive values such as strings and integers as keys. The precise requirement is called **hashability**; implementation details belong in a later data structures course.

Part 3: Controlling Loop Execution

Normal Iteration Runs the Body to Completion

In the loops from Lectures 22–24, each iteration normally follows the same control path.

bind next value $\longrightarrow$ **execute body** $\longrightarrow$ **request next value**

Sometimes the current value changes what should happen next.

- We may want to **skip the rest of this iteration**.
- We may know enough to **stop the loop entirely**.
- We may want special code to run only when the loop **finishes normally**.

Key idea

`continue`, `break`, and loop `else` refine the control flow of an existing `for` loop; they do not create a new kind of iteration.

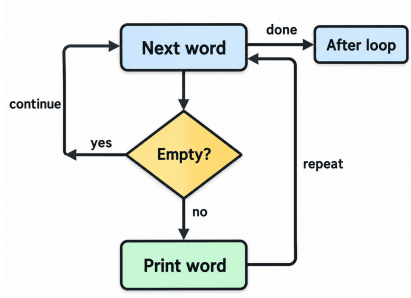
`continue`: Skip the Rest of This Iteration

Example

```
words = ["cat", "", "dog"]

for word in words:
    if word == "":
        continue
    print(word)
```

When `word == ""`, Python skips the remaining statements in the body and requests the next value.



Trace `continue`

Iteration	word	word == ""	Printed?
1	`"cat"`	False	`cat`
2	`""`	True	no
3	`"dog"`	False	`dog`

Key idea

`continue` ends only the **current iteration**. The loop itself remains active.

With `while`, `continue` Can Skip the Update

Both examples start with `values = [0, 5]`.

Incorrect: stuck at index 0

```
i = 0
while i < len(values):
   if values[i] == 0:
      continue
   print(values[i])
   i = i + 1
```

Advance before a possible skip

```
i = 0
while i < len(values):
   value = values[i]
   i = i + 1
   if value == 0:
      continue
   print(value)
```

The corrected loop prints 5 and stops. The left loop never reaches its update.

Key idea

`continue` returns to a `while` condition; it does not advance your index. A `for` loop requests its next item automatically.

Check the Boundary Before Reading the Element

Walk until a match or the end

```
values = [4, 9, 2]
target = 7
i = 0
while i < len(values) and values[i] != target:
   i = i + 1
```

- `and` checks its left side first. If it is false, Python skips the right side.
- At `i == len(values)`, the boundary test prevents an invalid lookup.
- Reversing the two tests can raise `IndexError`, including on an empty list.

In class

Final `i`: target 7 $\rightarrow$ 3; target 9 $\rightarrow$ 1; empty list $\rightarrow$ 0. Which results identify a valid matching index?

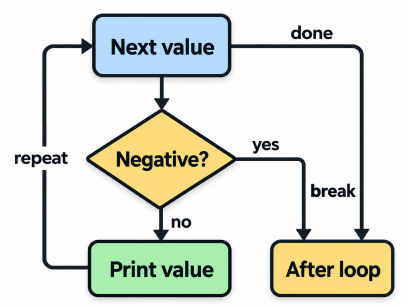
`break`: Terminate the Loop Immediately

Example

```
values = [4, 9, -2, 7]

for value in values:
    if value < 0:
        break
    print(value)
```

The loop prints 4 and 9. When -2 is encountered, the loop terminates; 7 is never bound to `value` by this loop.



`continue`, `break`, and `return` Are Different

Statement	Control effect
`continue`	stop the current iteration; request the next value
`break`	terminate the nearest loop
`return`	terminate the current function call

Tracing question

After the statement executes, ask: **where does control continue?** At the next iteration, after the loop, or back in the caller?

A `for` Loop Can Have an `else` Clause

The `else` clause runs only when the loop reaches its end **without executing** `break`.

Search pattern

```
target = 7
for value in values:
   if value == target:
      print("found")
      break
else:
   print("not found")
```

Key idea

The `else` belongs to the **loop**. It means "the loop exhausted its input without being terminated by `break`."

Reasoning About Loop `else`

Suppose `values = [4, 9, 2, 7]`.

Target	How loop ends	Does `else` run?
7	`break` executes	no
5	input is exhausted normally	yes

- `continue` does not suppress the loop `else`; the loop may still finish normally later.
- An empty input also completes normally, so the loop `else` runs.
- A `while` loop can also have an `else`: it runs when the condition becomes false without a `break`.

Why this form exists

It is useful when the interesting distinction is **found something and stopped** versus **examined everything and never found it**.

Part 4: Iterating over Dictionaries

A `for` Loop over a Dictionary Visits Keys

Example

```
scores = {"Ada":  88, "Grace":  94, "Linus":  79}

for name in scores:
   print(name)
```

During iteration, `name` is repeatedly bound to dictionary keys.

Key idea

The basic dictionary iteration model is: **one key per iteration**.

Use the Key to Retrieve Its Value

Code

```
for name in scores:
   score = scores[name]
   print(name, score)
```

Conceptually, each iteration does two things:

1. bind the next key to `name`,
2. use that key to retrieve its associated value.

key $\longrightarrow$ **dictionary lookup** $\longrightarrow$ **value**

Tracing Dictionary Iteration

Suppose:

Dictionary

```
scores = {"Ada":  88, "Grace":  94, "Linus":  79}
```

Iteration	name	scores[name]
1	"Ada"	88
2	"Grace"	94
3	"Linus"	79

The exact loop body determines what computation is performed with each association.

Explicit Key Iteration with `keys()`

Python also provides an explicit view of the keys.

Equivalent styles

```
for key in data:
    ...

for key in data.keys():
    ...
```

For ordinary key iteration, the shorter first form is usually sufficient.

Iterating over Values

Use `values()` when the keys themselves are not needed.

Example

```
for score in scores.values():
   print(score)
```

Repeated bindings are values rather than keys:

$$88 \rightarrow 94 \rightarrow 79$$

Iterating over Key–Value Pairs

Use `items()` when both parts of each association are needed.

Example

```
for name, score in scores.items():
   print(name, score)
```

Each iteration provides one key–value pair.

`("Ada", 88)` → `("Grace", 94)` → `("Linus", 79)`

Two Loop Variables Unpack One Supplied Pair

One pair, two bindings

```
name, score = ("Ada", 88)
```

In `for name, score in scores.items():`, each supplied pair is unpacked in the same way. It is still **one loop**.

Need a position and a value?

```
names = ["Ada", "Grace"]
for index, name in enumerate(names):
   print(index, name)
```

Output

```
0 Ada
1 Grace
```

Key idea

`items()` supplies key–value pairs; `enumerate()` supplies index–value pairs, starting at index `0` by default.

Updating Values Without Changing Membership

Replace list elements by index

```
values = [2, 5, 8]
for index, value in enumerate(values):
   values[index] = value * 2
```

The same list now contains `[4, 10, 16]`. Its length stays the same.

Replace values for existing keys

```
scores = {"Ada": 88, "Grace": 94}
for name in scores:
   scores[name] = scores[name] + 1
```

Result: `{"Ada": 89, "Grace": 95}`. No keys are added or removed.

Key idea

These statements assign through an index or key. Rebinding the loop variable alone would not replace a stored element. Existing aliases observe these intentional mutations.

Keep the Collection You Traverse Stable

Removing during list traversal

```
values = [0, 0, 1]
for value in values:
   if value == 0:
      values.remove(value)
```

Result: `[0, 1]`. The second zero shifts left and is skipped.

Build a separate result

```
values = [0, 0, 1]
kept = [value for value in values
          if value != 0]
```

`kept` is `[1]`; `values` remains `[0, 0, 1]`.

Key idea

Do not add or remove dictionary keys while iterating that dictionary; it can raise `RuntimeError`. Updating values for existing keys is allowed. Read one collection and build another when membership must change.

Another Accumulator: The Best Entry Seen So Far

Return the name with the highest score, or None for empty input

```
def highest_name(scores:  dict[str, int]):
   best = None
   for name, score in scores.items():
      if best is None or score > scores[best]:
         best = name
   return best
```

- `best` identifies the highest-scoring entry processed so far.
- `or` skips the lookup while `best is None`; the first entry establishes a real candidate.
- This handles negative scores too. Equal scores keep the first encountered name because the comparison uses `>`.

In class

Predict the result for `{"Ada": -8, "Grace": -2, "Linus": -2}` and for `{}`.

Choose the Iteration Form That Matches the Task

Need	Typical form
keys only	`for key in d:`
values only	`for value in d.values():`
both key and value	`for key, value in d.items():`

Key idea

Choose the view of the dictionary that directly supplies the information the loop body needs.

Association Is More Important Than Position

Modern Python dictionaries preserve insertion order, but their semantic purpose is still association by key.

- `d[0]` looks up the key `0`; it does not mean "first entry.
- We test membership with a key.
- We design dictionary algorithms around relationships between keys and values.

Design principle

Do not choose a dictionary because you need "the third item." Choose it because a meaningful key should identify a value.

Part 5: From Counting to a Frequency Map

Review: Counting One Character

With one target character, a single counter works well.

Count the letter `"a"`

```
count = 0
for char in word:
   if char == "a":
      count = count + 1
```

The state is one integer because we are answering one counting question.

What If We Want Every Character Count?

We could create one variable per possible character:

Poorly scaling idea

```
count_a = 0
count_b = 0
count_c = 0
...
```

But the set of characters appearing in the input is data. We should not need a separate variable written in advance for every possible character.

Key idea

We need a collection in which the **character itself** can identify its current count.

A Dictionary Can Store Many Counters

Let each character be a key and its observed frequency be the value.

Character key	Count value
`"b"`	1
`"a"`	3
`"n"`	2

For the word `"banana"`:

```
{"b":  1, "a":  3, "n":  2}
```

Frequency-Map Algorithm

Begin with no observed characters:

Initial state

```
counts = {}
```

For each character:

1. if the character is already a key, increase its count;
2. otherwise, create the key with initial count `1`.

Key idea

The dictionary grows only when a character is observed for the first time. Later occurrences update an existing association.

Trace: Building the Map for `"banana"`

Step	char	counts after iteration
1	`"b"`	`{"b": 1}`
2	`"a"`	`{"b": 1, "a": 1}`
3	`"n"`	`{"b": 1, "a": 1, "n": 1}`
4	`"a"`	`{"b": 1, "a": 2, "n": 1}`
5	`"n"`	`{"b": 1, "a": 2, "n": 2}`
6	`"a"`	`{"b": 1, "a": 3, "n": 2}`

new character $\rightarrow$ add key **seen character** $\rightarrow$ update value

The Complete Frequency-Map Function

Function

```
def frequency_map(word:  str) -> dict[str, int]:
   counts = {}
   for char in word:
      if char in counts:
         counts[char] = counts[char] + 1
      else:
         counts[char] = 1
   return counts
```

Key idea

This function combines iteration, conditionals, dictionary membership, dictionary lookup, and dictionary mutation into one coherent computation.

Prediction Challenge

Without running the code, predict:

Call

```
frequency_map("letter")
```

Determine:

1. the order in which characters are encountered,
2. when a new key is created,
3. when an existing value is updated,
4. the final key–value associations.

Do not skip the trace

The final dictionary is useful, but the important reasoning is how the state changes after each character.

Check the Meaning of the State, Including Empty Input

After each iteration of `frequency_map`, each stored count describes occurrences in the **prefix processed so far**.

- Before the loop, no characters have been processed: `counts = {}`.
- Processing one character increments exactly its count or starts it at `1`.
- At the end, the counts describe the entire input. The input string is unchanged.

Input	Expected result
`""`	`{}`
`"a"`	`{"a": 1}`
`"aaaa"`	`{"a": 4}`
`"letter"`	`{"l": 1, "e": 2, "t": 2, "r": 1}`

We iterate over the **word**, so growing the separate **counts** dictionary is safe.

Part 6: A Preview of the Data Structure Underneath

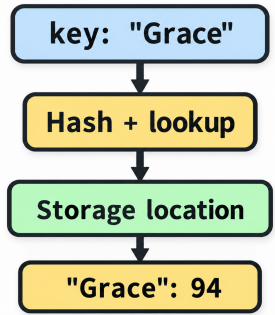
Why Dictionary Lookup Feels Different from List Search

A hash table uses a key to help locate its stored association.

- Lists fundamentally organize data by position.
- Dictionaries fundamentally organize data by key.
- Python dictionaries are implemented using a **hash-table** strategy.
- The key is processed to help determine where its association is stored.

Preview only

How hashing works, how collisions are handled, and why the structure is efficient are topics for a later data structures course.



Sequence or Dictionary?

Question	Sequence	Dictionary
How is data identified?	position	key
Typical access	`values[i]`	`data[key]`
Natural use	ordered data	named associations
Repeated processing	iterate values	iterate keys / values / pairs
Example	daily temperatures	character frequencies

Key idea

Choosing a data structure begins with identifying the relationship the program needs to represent.

Where This Course Stops

In CS 1001, you should be able to **use** a dictionary accurately:

- create key–value associations;
- retrieve and update values by key;
- test whether a key exists;
- iterate over keys, values, and pairs;
- use a dictionary to represent naturally associative data.

Later in data structures

You will ask how a hash table represents these associations internally, how operations are implemented, and how performance changes under different conditions.

Takeaways

1. Dictionaries organize data by **association** rather than numerical position.
2. Assignment through a key either adds a new association or updates an existing one.
3. `continue` skips the remainder of one iteration; `break` terminates the loop; loop `else` runs only after normal exhaustion.
4. Iterating over a dictionary naturally exposes its keys; `values()` and `items()` expose other useful views.
5. Guard lookups, preserve loop progress, and keep traversal sources stable; accumulated state can be counts or a best-so-far candidate.
6. The same tracing discipline still applies: follow bindings, control flow, mutations, and program state one step at a time.

Next

Lecture 26 moves from one structured collection to **nested data and two-dimensional lists**.

Optional Iteration Reference

`zip`: Traverse Two Sequences Together

Build associations from paired input

```
names = ["Ada", "Grace"]
marks = [88, 94]
scores = {}
for name, mark in zip(names, marks):
   scores[name] = mark
```

Result: `{"Ada": 88, "Grace": 94}`.

- Each iteration receives one pair: `("Ada", 88)`, then `("Grace", 94)`.
- This visits corresponding positions together; it does not visit every possible combination.
- By default, `zip` stops when the shortest input ends. Check equal lengths first if every value must have a partner.

Any and All Questions Have Different Early Exits

Does any value satisfy the condition?

```
def any_negative(values):
  for value in values:
    if value < 0:
      return True
  return False
```

One matching value proves `True`; `False` requires exhausting the input.

Do all values satisfy the condition?

```
def all_nonnegative(values):
  for value in values:
    if value < 0:
      return False
  return True
```

One counterexample proves `False`; `True` requires exhausting the input.

Key idea

On `[]`, these return `False` and `True`, respectively. "No value violates the rule" does not imply the input contained a value.

These are loop patterns; Python also provides the built-ins `any` and `all`.

Boundary Cases to Check for a Loop Function

- **No iterations:** empty input or an initially false `while` condition.
- **Exactly one iteration:** does initialization produce the right answer?
- **Conditional processing:** no matches, all matches, first match, and last match.
- **Extremum selection:** negative values and tied best values.
- **Early control transfer:** does each skip preserve progress, and where does each exit continue?
- **Input contract:** is the source left unchanged when a fresh result is promised?

Key idea

Trace the meaning of the state as well as the statements. A loop can terminate and still compute the wrong result.